Plan directeur des scans et rapports de sécurité — FileFlow

Titre	Plan directeur des scans, de la vérification et des rapports de sécurité
Produit	FileFlow — plateforme de partage de fichiers auto-hébergée
Version	1.1
Date	27 octobre 2026
Statut	Pour validation
Propriétaire	Driss OUAAZIZ — Software Architect
Révision de référence	`ca27c96` (branche `main`)
Classification	Interne

> Références : OWASP ASVS 5.0 (niveau 2) ; OWASP WSTG v4.2 ; OWASP Top 10 (2025) ; CWE Top 25 ; CVSS v4.0 ; NIST SP 800-218 (SSDF) ; SLSA v1.0 ; OpenSSF Scorecard. Documents liés : 06 (modèle de menaces), 13 (plan DAST OWASP ZAP), 14 (rapport d'audit d'octobre 2026).

Table des matières

1. Synthèse
2. Objet, périmètre et exclusions
3. État de départ
4. Stratégie de vérification en couches
5. Phase 1 — Cartographie de la surface d'attaque
6. Phase 2 — Revue de code manuelle par domaine
7. Phase 3 — Programme de scans automatisés
8. Phase 4 — Tests dynamiques et matrice d'autorisation
9. Phase 5 — Fuzzing et tests de propriétés
10. Phase 6 — Chaîne d'approvisionnement, build et CI
11. Phase 7 — Configuration de déploiement
12. Revue continue au poste de développement (plugin security-guidance)
13. Triage, sévérité et délais de remédiation
14. Gestion des exceptions et des faux positifs
15. Rapports de sécurité
16. Porte de sécurité (critères de mise en production)
17. Indicateurs de pilotage
18. Phasage et planning
19. Rôles et responsabilités
20. Risques de la campagne

- Annexe A — Inventaire des routes et matrice d'autorisation
- Annexe B — Fiche de constat (gabarit)
- Annexe C — Rapport de campagne (gabarit)
- Annexe D — Politique de sécurité propre au projet (proposition)
- Annexe E — Commandes de référence

---

1. Synthèse

Le rapport d'audit d'octobre 2026 (doc. 14) a montré que les vulnérabilités les plus graves de FileFlow — contournement du contrôle d'accès aux chemins, escalades via les partages — **n'étaient détectées par aucun scanner** : elles relèvent de la logique d'autorisation. Les six constats élevés et moyens (H1, H2, M1 à M4) sont corrigés et publiés, et l'ensemble des contrôles CI est vert sur `ca27c96`. Les cinq constats faibles (F1 à F5) ont été corrigés le 27 octobre 2026 ; restent ouverts les points N1 à N6 (§ 3.2).

Ce plan organise une vérification **exhaustive et reproductible** de la sécurité du produit, avec trois principes :

1. **Chaque propriété de sécurité est vérifiée par un test automatisé**, pas seulement constatée une fois. Un constat corrigé devient un test de non-régression exécuté en CI.
2. **Plusieurs couches indépendantes** : revue continue au poste de développement, contrôles bloquants sur chaque pull request, scans hebdomadaires, campagne approfondie trimestrielle et vérification avant chaque version.
3. **Une traçabilité de bout en bout** : chaque constat a un identifiant, une sévérité CVSS v4.0, un responsable, un délai, un test de non-régression et un statut publié dans un rapport.

Le livrable central de la campagne est la **matrice d'autorisation** (annexe A) : les 56 routes de l'API croisées avec chaque rôle et chaque relation de propriété, exécutée sous forme de tests d'intégration HTTP. C'est elle qui aurait détecté H1, H2 et M1.

---

2. Objet, périmètre et exclusions

2.1 Objet

Définir les activités de vérification de sécurité (revue, scans, tests, fuzzing), leur fréquence, leurs seuils bloquants, et le format des rapports produits.

2.2 Périmètre

COMPOSANT	CONTENU	VOLUME INDICATIF
Back-end	`backend/src` — Rust, axum 0.7, SQLite (rusqlite), JWT HS256, TOTP	~8 000 lignes, 19 modules
Front-end	`frontend/src` — React, TypeScript, Vite, DOMPurify, prévisualisations (Markdown, code, Word, Excel)	~40 composants
Image	`Dockerfile` multi-étapes, binaire musl statique, `distroless/static:nonroot`	11 Mo
Chaîne CI/CD	`.github/workflows/ci.yml`, `security.yml`, `dependabot.yml`	2 workflows, 12 jobs
Dépendances	`Cargo.lock`, `pnpm-lock.yaml`, actions GitHub, images de base	—
Configuration	Variables d'environnement, `docker-compose.yml`, documentation d'exploitation (doc. 10)	—

2.3 Exclusions

- Infrastructure d'hébergement de production (proxy inverse, pare-feu, système hôte) : traitée en § 11 sous forme de recommandations, pas de tests d'intrusion réseau.
- Tests de déni de service volumétriques (charge réseau) : seuls les DoS **applicatifs** (verrouillages, ressources, expressions régulières) sont testés.
- Ingénierie sociale et sécurité physique.

---

3. État de départ

3.1 Constats déjà corrigés

RÉF.	CONSTAT	COMMIT / DATE	VÉRIFICATION EXISTANTE
H1	Contournement par segments `..`	`de7a95e`	Tests unitaires + fuzz `resolve`
H2	Écriture via demande de dépôt public par un utilisateur en lecture seule	`945dc6c`	Script de reproduction local
M1	Escalade via dossiers partagés	`d3c6f42`	Script de reproduction local
M2	Remplacement du second facteur sans réauthentification	`ec0d17c`	Script de reproduction local
M3	Déconnexion en erreur 500	`c920a94`	Test unitaire
M4	Absence de limitation sur les liens publics	`c920a94`	Test unitaire + script
D1 à D4	Dépendances (`dompurify` , `zip` , `crypto-bigint` , `tokio`)	`759429f` , mises à jour	`trivy` , `cargo-audit` , `cargo-deny`
F1	Condition de concurrence sur les liens à téléchargement limité ; contournement du plafond par requêtes `Range` (découvert pendant la correction)	Corrigé le 27/10/2026	Tests unitaires (50 réservations concurrentes, restitution) + vérification sur instance locale
F2	Réutilisation d'un JWT après recréation d'un compte homonyme	Corrigé le 27/10/2026	Test unitaire + vérification sur instance locale
F3	Déni de service par verrouillage de compte (nouveau modèle : IP, couple utilisateur/IP, compte à 50 échecs)	Corrigé le 27/10/2026	Tests unitaires + vérification sur instance locale
F4	Rétrogradation ou désactivation du dernier administrateur actif	Corrigé le 27/10/2026	Test unitaire + vérification sur instance locale
F5	Liens publics et demandes de dépôt non rattachés au cycle de vie du fichier ; droits du créateur revérifiés à chaque téléchargement	Corrigé le 27/10/2026	Test unitaire + vérification sur instance locale

Écart principal : les vérifications de H2, M1, M2 et M4 reposent sur des scripts locaux non versionnés, et celles de F1 à F5 sur des tests unitaires complétés de vérifications manuelles sur une instance locale. Ils doivent devenir des tests d'intégration HTTP en CI (§ 8).

3.2 Constats ouverts

RÉF.	CONSTAT	SÉVÉRITÉ (DOC. 14)
N1	Verrouillage d'un lien public par un tiers (clé `link:<token>`)	À coter
N2	Image de build `node:20-slim` : Node.js 20 hors support depuis avril 2026	À coter
N3	Images de base référencées par étiquette et non par empreinte (`@sha256`)	À coter
N4	Chaîne d'outils Rust non figée en CI (`dtolnay/rust-toolchain@master` sans version)	À coter
N5	Lien public créable par le destinataire d'un partage interne (`POST /api/shares` n'exige que `validate_access(FileRead)`)	À coter — décision produit en attente
N6	Partage interne non revérifié contre les droits actuels de son émetteur (vérifié uniquement à la création)	À coter

3.3 Contrôles automatisés en place

`semgrep` (5 jeux de règles), ESLint (règles de sécurité, `no-unsanitized`), `clippy -D warnings` , `cargo-audit` , `cargo-deny` , `pnpm audit` , `trivy fs` , `trivy image` , `gitleaks` , `cargo-fuzz` (4 cibles, hebdomadaire), Dependabot (alertes, correctifs de sécurité automatiques, mises à jour hebdomadaires avec délai de carence de 7 jours).

Suites de tests : **103 tests back-end** (`cargo test`) et **114 tests front-end** (`pnpm test`), exécutés à chaque push et pull request.

4. Stratégie de vérification en couches

COUCHE	DÉCLENCHEUR	CONTRÔLES	BLOQUANT	TEMPS DE RETOUR
C0 — Poste de développement	Chaque modification, chaque commit	Plugin security-guidance (motifs dangereux, revue des diffs, revue au commit), `clippy`, `pnpm lint`	Non (signalement)	Secondes
C1 — Pull request	Chaque push / PR	Tests unitaires et d'intégration (dont matrice d'autorisation), SAST, SCA, secrets, licences, image	**Oui**	< 15 min
C2 — Hebdomadaire	Lundi 06:00 UTC	C1 complet sur `main` + fuzzing + DAST de base (ZAP baseline) + Scorecard	Alerte	< 1 h
C3 — Campagne trimestrielle	Planifiée	Revue manuelle complète (§ 6), DAST complet authentifié, fuzzing long, revue des exceptions	Rapport	1 à 2 semaines
C4 — Avant version	Étiquette `v*`	C1 + DAST actif + SBOM + signature + porte de sécurité (§ 16)	**Oui**	< 1 h
C5 — Événementielle	Nouvel avis critique, incident, changement d'architecture	Analyse d'atteignabilité, revue ciblée	Selon cas	< 48 h

Le principe est de **déplacer chaque contrôle vers la couche la plus précoce** où il reste fiable : un test d'autorisation écrit pendant la campagne C3 rejoint la couche C1.

---

5. Phase 1 — Cartographie de la surface d'attaque

Objectif : disposer d'un inventaire exhaustif de ce qui est exposé et de qui peut l'atteindre, avant toute revue.

ACTIVITÉ	MÉTHODE	LIVRABLE
Inventaire des routes	Extraction de `backend/src/lib.rs` (`.route(...)`, `.nest(...)`) et comparaison avec `documentation/08-api/openapi.yaml`	Annexe A complétée ; écarts OpenAPI ↔ code listés
Classification par zone de confiance	Anonyme / authentifié / administrateur / jeton public (`/s/`, `/r/`)	Colonne « zone » de l'annexe A
Identification du contrôle appliqué	Pour chaque gestionnaire : fonction de contrôle appelée (`validate_access`, `validate_own_access`, `own_grant`, `require_admin`…)	Colonne « contrôle » ; gestionnaires sans contrôle signalés
Flux de données sensibles	Mots de passe, secrets TOTP, codes de récupération, JWT, jetons de liens, contenu des fichiers	Diagramme de flux mis à jour dans le doc. 06
Frontières de confiance	Navigateur ↔ API, API ↔ système de fichiers, API ↔ SQLite, proxy inverse ↔ API (`TRUST_PROXY_HEADERS`), rendu client des fichiers utilisateurs	Liste des frontières et hypothèses
Mise à jour du modèle de menaces	STRIDE par frontière	Révision du doc. 06

Critère de sortie : 100 % des routes ont une zone, un contrôle identifié et au moins une ligne dans la matrice d'autorisation.

---

6. Phase 2 — Revue de code manuelle par domaine

La revue suit les chapitres de l'OWASP ASVS 5.0, niveau 2. Chaque point est soit **vérifié par un test** (référence du test), soit **justifié** (référence de code), soit ouvert en constat.

6.1 Authentification (ASVS V6) et sessions (V7, V9)

- Hachage bcrypt : coût, opération leurre contre l'énumération, temps de réponse constant (mesure statistique sur 1 000 essais, utilisateur existant contre inexistant).
- JWT : algorithme figé à HS256 (rejet de `none`, `RS256`, `HS512`), vérification de `exp`, `tv` (version de jeton), origine et entropie du secret, absence du secret dans les journaux.
- **F2** : identité immuable dans les claims.
- Cookies : `HttpOnly`, `Secure`, `SameSite=Strict`, `Path`, durée ; suppression complète à la déconnexion (M3).
- 2FA : configuration, confirmation, désactivation, régénération des codes (M2) ; usage unique des codes de récupération ; fenêtre TOTP et rejeu d'un même code ; limitation des tentatives.
- Verrouillage (**F3**, corrigé) : vérifier le modèle en place — par IP (5 échecs), par couple utilisateur + IP (5 échecs), par compte toutes IP (50 échecs, compteur non remis à zéro par une connexion réussie) — et l'absence de contournement par rotation d'IP ou de nom d'utilisateur.
- Révocation : changement de mot de passe, « déconnecter partout », modification des droits par un administrateur.

6.2 Autorisation (ASVS V8)

- Chaque route de l'annexe A passe par un contrôle explicite ; aucune décision d'accès côté client uniquement.
- Accès horizontaux (IDOR) sur les identifiants : liens publics, demandes de dépôt, dossiers partagés, partages internes, favoris, versions, corbeille.
- Accès verticaux : routes `/admin/*` refusées aux non-administrateurs ; **F4** (dernier administrateur).
- Autorité dérivée : partages internes et dossiers partagés ne donnent jamais plus que les droits propres du créateur (M1), y compris après révocation de ces droits.
- **F5** : cycle de vie des liens publics lors de `rename`, `move`, `delete` et du retrait des droits du créateur.

6.3 Fichiers et chemins (ASVS V5)

- Normalisation (`normalize_virtual`, `path_covers`) : `..`, `.`, barres multiples, barres inverses, encodage URL simple et double, octet NUL, Unicode (normalisation NFC/NFD, caractères de largeur nulle), casse.
- Liens symboliques dans `ROOT_DIR`, y compris créés après la vérification (fenêtre TOCTOU).
- Répertoire interne `.fileflow*` inaccessible à toute profondeur.
- Archives ZIP (`download-bulk`) : noms d'entrées, chemins absolus, consommation mémoire, nombre de fichiers.
- En-tête `Content-Disposition` : injection de retours à la ligne, encodage RFC 5987.
- Types MIME : HTML, SVG, XML et PDF jamais servis en ligne avec un type actif.

6.4 Téléversements

- Téléversement par morceaux : propriété de la session de téléversement, ordre et taille des morceaux, reprise, abandon, nettoyage des fichiers temporaires.
- Quotas : condition de concurrence entre requêtes parallèles, quotas des dépôts anonymes (`PUBLIC_UPLOAD_MAX_MB`).
- Épuisement disque : comportement quand le volume est plein (pas de fichier partiel visible, pas de panique).

6.5 Liens publics

- Entropie et format des jetons ; absence des jetons dans les journaux et l'en-tête `Referer`.
- Expiration, mot de passe, limite de téléchargements (**F1**), y compris par requêtes partielles `Range` (ignorées sur les liens plafonnés).
- Création d'un lien public par le destinataire d'un partage interne (**N5** : décision à documenter).
- Verrouillage par lien et par IP (**N1** : décision de conception à documenter dans un ADR).

6.6 Sécurité navigateur (ASVS V3) et API (V4)

- En-têtes : CSP (pas de `unsafe-inline` pour les scripts), `frame-ancestors`, HSTS, COOP, CORP, `Permissions-Policy`, `Referrer-Policy`, `nosniff`.

- CSRF : `SameSite=Strict` + méthodes non sûres uniquement en POST/PUT/DELETE ; aucune action d'état en GET.
- CORS : absence de politique permissive.
- Cache : `Cache-Control: no-store` sur les réponses sensibles (`/auth/me` , fichiers privés, codes de récupération).
- XSS dans les prévisualisations (Markdown, code, Word, Excel) : chaque `dangerouslySetInnerHTML` passe par DOMPurify ; noms de fichiers et métadonnées échappés.
- Expressions régulières exposées à des entrées utilisateur (ReDoS) côté client et serveur.

6.7 Déni de service applicatif

- Limites de taille de corps par route, nombre de champs multipart, longueur des noms.
- Recherche de fichiers et recherche par étiquette : coût sur une arborescence volumineuse.
- Journal d'audit : rotation et coût des requêtes (`limit` plafonné à 5 000).
- Concurrence des téléchargements groupés (sémaphore).

6.8 Données, secrets, journalisation (ASVS V11, V14, V16)

- Permissions des fichiers SQLite, de configuration et de données dans le conteneur.
- Génération et stockage du secret JWT et des secrets TOTP.
- Journal d'audit : exhaustivité des événements de sécurité (connexion, échec, 2FA, partage, administration), absence de secrets, intégrité, résistance à l'injection CSV.
- Messages d'erreur : pas de chemins système, de traces ni de détails SQL renvoyés au client.

Critère de sortie : chaque point est marqué « testé », « justifié » ou « constat ».

---

7. Phase 3 — Programme de scans automatisés

7.1 Outils en place

OUTIL	CATÉGORIE	CIBLE	COUCHE	SEUIL BLOQUANT
`semgrep` (OWASP Top 10, Rust, TypeScript, React, secrets)	SAST	Code	C1	Tout constat
ESLint + `eslint-plugin-security` + `no-unsanitized`	SAST front-end	`frontend/src`	C1	Toute erreur
`clippy` (`unwrap_used` , `-D warnings`)	Qualité / robustesse	`backend`	C1	Tout avertissement
`cargo-audit`	SCA Rust	`Cargo.lock`	C1 + C2	Tout avis non justifié
`cargo-deny`	SCA, licences, sources, crates bannies	`Cargo.lock`	C1	Toute violation
`pnpm audit`	SCA JavaScript	`pnpm-lock.yaml`	C1	Gravité ≥ modérée
`trivy fs`	Vulnérabilités, secrets, mauvaises configurations	Dépôt	C1	Gravité ≥ élevée
`trivy image`	Vulnérabilités de l'image	Image construite	C1	Gravité ≥ élevée
`gitleaks`	Secrets	Historique Git complet	C1	Tout secret
`cargo-fuzz`	Fuzzing	4 cibles	C2	Tout plantage
Dependabot	Mises à jour	Cargo, npm, actions, Docker	Continu	—

7.2 Outils à ajouter

OUTIL	APPORT	COUCHE	PRIORITÉ
Tests d'intégration d'autorisation (`backend/tests/`)	Détecte les failles de logique d'accès, que les scanners ne voient pas	C1	**P0**
CodeQL (JavaScript/TypeScript, Rust)	Analyse de flux de données inter-procédurale, complémentaire de semgrep	C1	P1
OWASP ZAP (doc. 13) — baseline puis scan complet authentifié	DAST : en-têtes, cookies, injections, comportements à l'exécution	C2 (baseline), C3/C4 (complet)	P1
zizmor	Audit des workflows GitHub Actions (injection de modèles, permissions excessives, `pull_request_target`)	C1	P1
OpenSSF Scorecard	Score de maturité de la chaîne d'approvisionnement	C2	P2
syft (SBOM CycloneDX) + **grype**	Inventaire publié avec chaque version, re-scan continu de l'inventaire	C4	P2
cosign + attestation de provenance SLSA	Intégrité et origine de l'image	C4	P2
proptest (Rust) / **fast-check** (TypeScript)	Tests de propriétés sur la normalisation des chemins, le Markdown et la sanitisation	C1	P2
hadolint	Bonnes pratiques Dockerfile (complète `trivy` misconfig)	C1	P3

7.3 Publication des résultats

Tous les outils capables de produire du **SARIF** (semgrep, CodeQL, trivy, zizmor, gitleaks) publient leurs résultats dans l'onglet *Code scanning* de GitHub. Cela donne un historique, un suivi par constat et des annotations directement dans les pull requests.

---

8. Phase 4 — Tests dynamiques et matrice d'autorisation

8.1 Matrice d'autorisation exécutable

Livrable principal de la campagne. Une suite de tests d'intégration Rust (`backend/tests/authz_matrix.rs`) démarre une instance sur un répertoire temporaire, crée un jeu de données déterministe, puis exécute chaque ligne de l'annexe A.

Acteurs de test :

ACTEUR	RÔLE	PARTICULARITÉ
`anon`	Non authentifié	—
`admin`	Administrateur	—
`alice`	Éditrice	Propriétaire de `/alice` , crée partages et liens
`bob`	Éditeur	Restreint à `/teamA` par groupe
`carol`	Lecteur	Lecture seule
`dave`	Éditeur désactivé	Jeton émis avant la désactivation
`eve`	Éditrice	Membre d'un dossier partagé d' `alice`

Dimensions croisées : route × acteur × relation à la ressource (propriétaire, membre, tiers, inexistante) × variante de chemin (normal, `..` , encodé, symlink). Résultat attendu : code HTTP **et** absence d'effet de bord (le fichier n'a pas changé, aucun lien n'a été créé).

Scénarios de non-régression obligatoires : H1, H2, M1 à M4, F1 à F5, chacun sous forme de test nommé (`regression_h1_dotdot_group_escape` , etc.).

8.2 DAST avec OWASP ZAP

Le doc. 13 décrit la campagne ZAP en détail (environnement, amorçage, politiques de scan, triage). Ce plan fixe la fréquence :

SCAN	COUCHE	DURÉE	SEUIL
Baseline passif	C2	~10 min	Aucune alerte « High »
API (OpenAPI) authentifié	C3, C4	~30 min	Aucune alerte « High » ou « Medium » non triée
Actif complet, un contexte par rôle	C3, C4	1 à 2 h	Idem

8.3 Scénarios ciblés manuels

Tests difficiles à automatiser dans ZAP, exécutés à chaque campagne C3 et transformés en tests dès que possible :

- Concurrence : 50 téléchargements parallèles d'un lien à usage unique (F1) ; quotas sous téléversements parallèles.
- Rejeu : réutilisation d'un code TOTP dans la même fenêtre ; réutilisation d'un code de récupération ; JWT après suppression puis recréation du compte (F2).
- Verrouillages : peut-on bloquer un compte (F3) ou un lien (N1) d'un tiers ?
- Cycle de vie : lien public après renommage, déplacement, suppression puis recréation du fichier (F5).
- Proxy : en-têtes `X-Forwarded-For` falsifiés avec `TRUST_PROXY_HEADERS=false` et `true`.
- Navigateur : fichiers HTML, SVG et Markdown piégés ouverts en prévisualisation et via lien public.

---

9. Phase 5 — Fuzzing et tests de propriétés

CIBLE	ÉTAT	INVARIANT VÉRIFIÉ	DURÉE C2	DURÉE C3
`resolve`	En place	Le chemin résolu reste sous la racine	60 s	1 h
`sanitize_name`	En place	Aucun séparateur ni caractère de contrôle	60 s	1 h
`parse_range`	En place	Plages valides et bornées	60 s	1 h
`content_disposition`	En place	Pas de retour à la ligne ni de guillemet non échappé	60 s	1 h
`normalize_virtual` + `path_covers`	**À créer**	Si `path_covers(a, b)` alors `b` résolu est sous `a` résolu	60 s	1 h
Multipart public (`/r/`)	**À créer**	Aucun fichier écrit sans mot de passe valide	60 s	1 h
Parsing JWT et cookies	**À créer**	Jamais accepté sans signature valide	60 s	1 h
`parseAtxHeading` et rendu Markdown (fast-check)	**À créer**	Temps linéaire, sortie sans balise active après DOMPurify	CI	CI

Les corpus sont versionnés dans `backend/fuzz/corpus/` ; tout plantage trouvé devient un cas du corpus et un test unitaire.

---

10. Phase 6 — Chaîne d'approvisionnement, build et CI

POINT	CONSTAT / ACTION	RÉFÉRENCE
Image de build Node.js	Passer de `node:20-slim` (hors support) à une LTS active (22 ou 24) (**N2**)	Dockerfile
Empreintes des images	Référencer `node`, `rust` et `distroless` par `@sha256:` ; Dependabot (écosystème `docker`) les maintient à jour (**N3**)	Dockerfile

POINT	CONSTAT / ACTION	RÉFÉRENCE
Chaîne d'outils Rust	Fichier `rust-toolchain.toml` avec version figée, lu par la CI et le Dockerfile (**N4**)	`backend/`
Actions GitHub	Déjà épinglées par empreinte de commit — à maintenir ; audit `zizmor`	Workflows
Permissions des workflows	`contents: read` par défaut ; permissions élargies uniquement au niveau du job (`security-events: write` pour SARIF)	Workflows
Protection de branche	`main` : PR obligatoire, contrôles C1 requis, historique linéaire, pas de push forcé	Paramètres du dépôt
Fonctions GitHub	*Secret scanning* + *push protection*, *private vulnerability reporting*, alertes Dependabot (actives)	Paramètres du dépôt
Politique de sécurité	Fichier `SECURITY.md` (signalement, délais de réponse, versions supportées)	Racine
Dependabot	Fusion des PR groupées mineures/correctives après CI verte ; mises à jour majeures traitées une à une avec revue	`dependabot.yml`
SBOM et signature	SBOM CycloneDX, signature `cosign` sans clé, attestation de provenance à chaque version	C4
Reproductibilité	`--locked` / `--frozen-lockfile` partout (en place)	CI, Dockerfile

11. Phase 7 — Configuration de déploiement

PARAMÈTRE	VÉRIFICATION	VALEUR RECOMMANDÉE
`TRUST_PROXY_HEADERS`	N'est activé que derrière un proxy qui **écrase** `X-Forwarded-For`	`false` par défaut
TLS	Terminé au proxy ; TLS 1.2 minimum, 1.3 recommandé	—
`COOKIE_SECURE` / HSTS	Actifs en production	`true`
`MAX_UPLOAD_MB`, **`PUBLIC_UPLOAD_MAX_MB`**	Documentés dans le doc. 10 ; le second ≤ le premier	200 / 100 (défauts)
Secret JWT	Fourni ou généré, ≥ 256 bits, jamais journalisé, rotation documentée	—
Conteneur	Utilisateur non-root (en place), système de fichiers en lecture seule hors `/data` et `/config`, `no-new-privileges`, capacités supprimées	`docker-compose.yml`
Sauvegardes	Chiffrement et contrôle d'accès des sauvegardes de `/data` et de SQLite	Doc. 10
Journalisation	Rétention et protection du journal d'audit	Doc. 10 et 11

Un contrôle `trivy config` sur `docker-compose.yml` et un test de démarrage avec `--read-only` sont ajoutés à la couche C1.

12. Revue continue au poste de développement (plugin security-guidance)

Le plugin **security-guidance** est installé dans l'environnement de développement. Il agit avant même la pull request, en trois niveaux :

NIVEAU	DÉCLENCHEUR	CE QUI EST VÉRIFIÉ	USAGE DANS FILEFLOW
1 — Motifs	Chaque écriture de fichier	~25 motifs dangereux connus : `innerHTML` brut, secrets en dur, désérialisation non sûre…	Alerte immédiate sur les composants de prévisualisation
2 — Revue des diffs	Fin de chaque session de modification	Injection, XSS, SSRF, IDOR, contournement d'authentification, traversée de chemin, sur les lignes modifiées	Constats remontés et corrigés avant le commit

NIVEAU	DÉCLENCHEUR	CE QUI EST VÉRIFIÉ	USAGE DANS FILEFLOW
3 — Revue au commit	`git commit`	Traçage du flux de données à travers plusieurs fichiers (IDOR, autorisation inter-modules)	Détecte les failles du type M1, où la faille naît de l'interaction entre deux modules

12.1 Politique propre au projet

Le plugin accepte un fichier de règles propres au projet, versionné avec le code et pris en compte à chaque revue des diffs (8 Ko maximum). L'annexe D propose son contenu : il encode les invariants de FileFlow (normalisation des chemins, autorité dérivée, verrouillages, rendu des fichiers utilisateurs) que les règles génériques ne peuvent pas deviner.

12.2 Limites et articulation

- Les constats du plugin sont des **signalements**, pas une preuve : ils sont triés comme ceux de tout autre outil (§ 13).
- Le plugin ne remplace ni la CI (couche C1), ni la matrice d'autorisation, ni la revue manuelle. Il réduit le nombre de défauts qui atteignent ces couches.
- Le niveau 3 ne lit pas le fichier de politique projet : les invariants critiques doivent donc aussi être couverts par des tests.
- Les revues transmettent les diffs au service de revue configuré ; aucun secret ne doit figurer dans le fichier de politique.
- Coût : chaque revue a un coût d'exécution, consigné par le plugin dans son journal local et suivi mensuellement.

12.3 Bilan observé

Pendant la correction des constats d'octobre 2026, le niveau 2 a remonté deux points pertinents : un défaut de comptabilisation de quota sur le chemin d'erreur des dépôts publics (corrigé dans `c920a94`) et le risque de verrouillage d'un lien par un tiers (N1, ouvert).

---

13. Triage, sévérité et délais de remédiation

13.1 Cotation

Chaque constat est coté en **CVSS v4.0** (vecteur complet consigné), puis ajusté selon l'**atteignabilité** dans FileFlow :

ATTEIGNABILITÉ	EFFET SUR LA PRIORITÉ
Atteignable depuis une zone anonyme	Priorité relevée d'un cran
Atteignable par un utilisateur authentifié	Priorité inchangée
Non atteignable (code mort, fonction non utilisée)	Exception justifiée (§ 14), mise à jour au prochain cycle

13.2 Délais de remédiation

SÉVÉRITÉ	CVSS V4.0	DÉLAI DE CORRECTION	DÉLAI DE TEST DE NON-RÉGRESSION
Critique	9,0 – 10,0	48 heures	Avec le correctif
Élevée	7,0 – 8,9	7 jours	Avec le correctif
Moyenne	4,0 – 6,9	30 jours	Avec le correctif
Faible	0,1 – 3,9	90 jours	Avec le correctif
Information	—	Prochain cycle	—

13.3 Cycle de vie d'un constat

`Nouveau → Trié → Confirmé (reproduction) → En correction → Corrigé (commit + test) → Vérifié (CI verte, rejouer la reproduction) → Clos`, ou `Trié → Faux positif` / `Risque accepté` (avec justification et échéance).

---

14. Gestion des exceptions et des faux positifs

MÉCANISME	FICHIER	RÈGLE
Avis RustSec ignorés	`backend/.cargo/audit.toml`, `backend/deny.toml` (miroirs)	Justification d'atteignabilité obligatoire en commentaire
Exclusions semgrep	`// nosemgrep` sur la ligne au-dessus	Commentaire expliquant la sanitisation ou la non-atteignabilité
Exclusions ESLint	`// eslint-disable-line <règle>`	Idem ; jamais de désactivation globale d'une règle de sécurité
Constats ZAP	Journal de triage (doc. 13, annexe C)	Identifiant de règle + URL + justification
Risques acceptés	Doc. 06, § 9 et ADR	Décision datée, propriétaire, **date de réexamen**

Registre des exceptions : chaque campagne C3 liste toutes les exceptions en vigueur et réexamine celles dont la date est échue. Exceptions actuelles : RUSTSEC-2023-0071 et RUSTSEC-2026-0387 (chiffrement RSA/EC inutilisé, HS256 seul — ADR 0003), exclusions DOMPurify documentées dans `PreviewPanel.tsx` et `markdown.ts`.

---

15. Rapports de sécurité

15.1 Types de rapports

RAPPORT	FRÉQUENCE	CONTENU	FORMAT	DESTINATAIRES	CLASSIFICATION
Résultats CI	Chaque exécution	Statut par job, constats SARIF	GitHub (*Code scanning*, annotations de PR)	Développeurs	Interne
Bulletin hebdomadaire	Lundi	Nouveaux avis, PR Dependabot, résultats fuzzing et ZAP baseline, constats ouverts et délais	Markdown (1 page)	Propriétaire	Interne
Rapport de campagne	Trimestriel	Revue complète, matrice, DAST, constats détaillés, preuves, plan de remédiation	Markdown + PDF + DOCX, format du doc. 14 (annexe C)	Propriétaire, exploitation	**Confidentiel**
Attestation de version	Chaque version	Porte de sécurité (§ 16) cochée, SBOM, signature, exceptions en vigueur	Markdown joint à la *release*	Exploitants, utilisateurs	Public (sans détails d'exploitation)
Rapport d'incident	Sur incident	Chronologie, impact, cause, correctif, actions préventives	Markdown + PDF	Propriétaire	Confidentiel

15.2 Règles de rédaction

- Chaque constat suit la fiche de l'annexe B : identifiant, sévérité CVSS, emplacement (`fichier:ligne`), scénario reproductible, impact, correctif, test de non-régression.
- Les preuves sont produites sur une instance **jetable** avec des données de test ; aucune donnée réelle dans un rapport.
- Les rapports confidentiels (scénarios d'exploitation) ne sont **pas versionnés** dans le dépôt public tant que les constats ne sont pas corrigés ; une version expurgée peut l'être ensuite.
- Le statut des constats est tenu à jour dans le dernier rapport de campagne (colonne « Statut ») plutôt que dans de nouveaux documents.

15.3 Archivage

Rapports de campagne et attestations conservés 3 ans ; artefacts CI (SARIF, SBOM, journaux de fuzzing) conservés 90 jours par GitHub et archivés avec chaque version.

---

16. Porte de sécurité (critères de mise en production)

Une version n'est publiée que si **tous** les critères sont satisfaits :

1. Tous les jobs `ci` et `security` sont verts sur le commit étiqueté.
2. Aucun constat critique ou élevé ouvert ; aucun constat moyen hors délai.
3. Matrice d'autorisation : 100 % des routes couvertes, 100 % des tests réussis.
4. Scan ZAP API authentifié : aucune alerte « High », alertes « Medium » toutes triées.
5. Fuzzing : aucun plantage non résolu sur la dernière exécution longue.
6. Toutes les exceptions ont une justification et une date de réexamen non échue.
7. SBOM généré, image signée, attestation de version rédigée.
8. Documentation d'exploitation (doc. 10) à jour des nouveaux paramètres de sécurité.

---

17. Indicateurs de pilotage

INDICATEUR	CIBLE
Délai moyen de correction (MTTR) par sévérité	Dans les délais du § 13.2
Constats ouverts hors délai	0
Couverture de la matrice d'autorisation (routes couvertes / routes totales)	100 %
Constats corrigés disposant d'un test de non-régression	100 %
Exceptions échues	0
Âge maximal d'une PR Dependabot de sécurité	7 jours
Score OpenSSF Scorecard	≥ 7/10
Vulnérabilités de l'image (`trivy image`)	0 élevée/critique
Exécutions de fuzzing cumulées par cible et par trimestre	≥ 10^8

---

18. Phasage et planning

SEMAINE	ACTIVITÉS	LIVRABLES
S1	Phase 1 (cartographie) ; politique security-guidance (annexe D) ; N2–N4 (Node LTS, empreintes, chaîne Rust)	Annexe A complétée, doc. 06 révisé, PR build
S2	Matrice d'autorisation exécutable (§ 8.1) ; tests de non-régression H1–M4	`backend/tests/authz_matrix.rs` en CI
S2–S3	Phase 2 (revue manuelle), tests d'intégration F1–F5 (corrigés le 27/10/2026), décisions N1 et N5 (ADR)	Constats, correctifs, tests
S3	CodeQL, zizmor, SARIF, protection de branche, `SECURITY.md`	Workflows mis à jour
S3–S4	Nouvelles cibles de fuzzing, tests de propriétés, fuzzing long	Corpus, rapport de fuzzing
S4	DAST ZAP (doc. 13) ; scénarios ciblés (§ 8.3)	Journal de triage
S4	SBOM, signature, attestation de version	Pipeline de publication
S5	Rapport de campagne, mise à jour des docs 06, 10, 12 et 14 ; porte de sécurité	Rapport PDF/DOCX, version publiée

Ensuite : bulletin hebdomadaire (C2) et campagne trimestrielle (C3), la prochaine en janvier 2027.

---

19. Rôles et responsabilités

ACTIVITÉ	RESPONSABLE SÉCURITÉ APPLICATIVE	DÉVELOPPEMENT	EXPLOITATION
Cartographie, modèle de menaces	R/A	C	I
Revue manuelle	R/A	C	—
Outils et CI	A	R	I
Correction des constats	A	R	I
DAST et tests de déploiement	R/A	C	C
Rapports	R/A	C	I
Porte de sécurité	A	R	C

R : réalise — A : approuve — C : consulté — I : informé. Le rôle de responsable sécurité applicative est tenu par le propriétaire du produit.

---

20. Risques de la campagne

RISQUE	MITIGATION
Faux sentiment de sécurité dû à une CI verte	La matrice d'autorisation et la revue manuelle sont obligatoires ; une CI verte n'est qu'un critère parmi huit (§ 16)
Bruit des outils menant à l'ignorance des alertes	Seuils bloquants ciblés, triage systématique, exceptions datées
Scan actif ZAP altérant des données	Instance jetable uniquement, jamais la production (doc. 13)
Fuite d'un rapport confidentiel	Rapports non versionnés tant que les constats sont ouverts ; diffusion nominative
Mises à jour majeures cassantes (axum 0.8, React, Tailwind 4…)	Traitées une à une, hors campagne, avec la matrice d'autorisation comme filet
Coût des revues automatisées	Suivi mensuel du journal du plugin ; modèle plus économique si nécessaire

---

Annexe A — Inventaire des routes et matrice d'autorisation

Zones : **Anon** (anonyme), **Auth** (authentifié), **Admin**, **Jeton** (jeton de lien public). Colonnes d'acteurs : résultat attendu (code HTTP). « P » = propriétaire ou droit accordé, « T » = tiers sans droit. À compléter en phase 1 pour chaque route.

A.1 Routes anonymes et à jeton

MÉTHODE	ROUTE	ZONE	CONTRÔLE	ANON	JETON VALIDE	JETON INVALIDE / EXPIRÉ
GET	`/api/health`	Anon	—	200	—	—
POST	`/api/auth/login`	Anon	Limiteur utilisateur + IP	200/401/429	—	—
POST	`/api/auth/logout`	Anon	— (efface les cookies)	200	—	—
POST	`/api/auth/2fa/verify`	Anon	Jeton intermédiaire 2FA + limiteur	401	200	401/429
POST	`/api/auth/2fa/verify-recovery`	Anon	Idem	401	200	401/429
GET/POST	`/s/:token`	Jeton	Mot de passe, expiration, limite, verrouillage	—	200	404/401/429
GET/POST	`/r/:token`	Jeton	Mot de passe d'abord, droits du créateur, plafond	—	200	404/401/429

A.2 Routes authentifiées (extrait — 41 routes)

MÉTHODE	ROUTE	CONTRÔLE	ANON	CAROL (LECTURE)	BOB (RESTREINT) P / T	ALICE P / T	ADMIN
GET	`/api/files`	`validate_access(Read)`	401	200	200 / 403	200 / 403	200
GET	`/api/files/download`	`validate_access(Read)`	401	200	200 / 403	200 / 403	200
GET	`/api/files/download-bulk`	Par élément	401	200	200 / 403	200 / 403	200
POST	`/api/files/upload`	`validate_access(Write)`	401	403	200 / 403	200 / 403	200
PUT	`/api/uploads/chunk`	Propriété de la session	401	403	200 / 404	200 / 404	200
POST	`/api/files/rename`, `/move`, `/delete`, `/mkdir`	`validate_access(Write)` source **et** cible	401	403	200 / 403	200 / 403	200
POST	`/api/shares`	`validate_own_access(Share)`	401	403	200 / 403	200 / 403	200
DELETE	`/api/shares/:token`	Créateur ou admin	401	404	200 / 404	200 / 404	200
POST	`/api/shared-folders`	`own_grant(Read)`	401	200 / 403	200 / 403	200 / 403	200
PUT	`/api/shared-folders/:id/members`	Créateur + `own_grant(Write)` si écriture	401	403	200 / 404	200 / 404	200
POST	`/api/upload-requests`	`validate_own_access(Write)`	401	403	200 / 403	200 / 403	200
POST	`/api/auth/2fa/disable`	Mot de passe + second facteur	401	200/401	200/401	200/401	200/401
…	(31 autres routes)						

Variantes de chemin à appliquer à chaque route prenant un chemin : normal, `a/../b`, `%2e%2e/`, `%252e%252e/`, `\..\`, `a//b`, NUL, symlink sortant, `.fileflow/`.

A.3 Routes d'administration (`/api/admin/*`, 8 routes)

Attendu : `anon` → 401, tout non-administrateur → 403, `admin` → 200, avec les gardes métier (dernier administrateur — F4, auto-suppression).

Annexe B — Fiche de constat (gabarit)

CHAMP	CONTENU
Identifiant	`AAAA-MM-<H/M/F/N><n>`
Titre	Une phrase décrivant le défaut
Sévérité	CVSS v4.0 : score + vecteur ; atteignabilité
Référentiel	ASVS (exigence), CWE, OWASP Top 10
Emplacement	`fichier:ligne`, commit audité
Préconditions	Rôle, configuration, données nécessaires
Scénario	Étapes reproductibles sur instance jetable

CHAMP	CONTENU
Résultat observé / attendu	—
Impact	Confidentialité, intégrité, disponibilité
Correctif	Description et commit
Test de non-régression	Nom du test et fichier
Statut et dates	Nouveau → Clos, avec dates

Annexe C — Rapport de campagne (gabarit)

1. Synthèse (tableau des constats par sévérité, faits marquants, tendance par rapport à la campagne précédente)
2. Périmètre, révision auditée, méthode et outils (versions)
3. Constats détaillés (fiches de l'annexe B)
4. Résultats des outils automatisés (tableau par outil, différences avec la campagne précédente)
5. Couverture : matrice d'autorisation, fuzzing (exécutions, couverture), DAST (URL explorées, règles actives)
6. Preuves de reproduction
7. Exceptions en vigueur et réexamens
8. Points forts constatés
9. Plan de remédiation (priorité, effort, échéance)
10. Indicateurs (§ 17)

Annexe D — Politique de sécurité propre au projet (proposition)

Fichier de règles du plugin, à placer dans le répertoire de configuration du projet (versionné) :

```
# FileFlow — règles de sécurité propres au projet

## Chemins et accès aux fichiers
- Tout chemin issu d'une requête passe par files::normalize_virtual avant
  toute comparaison ; ne jamais comparer des chemins bruts avec starts_with.
  Utiliser files::path_covers pour tester qu'un chemin est sous un autre.
- Tout gestionnaire qui lit ou écrit un fichier appelle validate_access avec
  la permission exacte (FileRead, FileWrite, Share...). Une opération à deux
  chemins (move, rename, copy) vérifie la source ET la destination.
- Ne jamais ouvrir un chemin sans files::resolve (protection symlink et
  blocage du répertoire .fileflow).

## Autorité dérivée
- Un partage interne, un dossier partagé, un lien public ou une demande de
  dépôt ne donne jamais plus que les droits propres de son créateur :
  vérifier avec files::own_grant / validate_own_access (qui excluent les
  accès obtenus par partage), à la création ET à chaque utilisation.

## Authentification et sessions
- JWT : HS256 uniquement ; toute nouvelle claim d'identité est vérifiée
  contre token_version. Toute modification de facteur d'authentification
  incrémente token_version (rotate_session).
- Changer ou désactiver la 2FA exige mot de passe + second facteur.
- Les cookies de session restent HttpOnly, Secure, SameSite=Strict.

## Points publics (/s/, /r/)
- Appliquer link_limiter (clé lien + clé IP) avant toute vérification de
  mot de passe ; le mot de passe est validé avant d'accepter un fichier.
- Les fichiers utilisateurs ne sont jamais servis en ligne avec un type
```

```
  actif (HTML, SVG, XML) : Content-Disposition attachment ou text/plain.
- L'IP client provient de audit::client_ip ; ne jamais lire
  X-Forwarded-For directement.

## Front-end
- Tout dangerouslySetInnerHTML reçoit une sortie de sanitize.ts (DOMPurify).
- Pas de RegExp construite à partir d'une entrée utilisateur ; les motifs
  appliqués au contenu des fichiers doivent être linéaires.

## Robustesse
- Pas de unwrap()/expect() hors tests sur des données externes : le binaire
  de production est compilé avec panic=abort.
```

Annexe E — Commandes de référence

```
# Couche C1 en local
cd backend && cargo test --locked && cargo clippy --all-targets --locked -- -D warnings
cargo audit && cargo deny check
cd ../frontend && pnpm test && pnpm lint && pnpm audit --audit-level moderate
semgrep --config p/owasp-top-ten --config p/rust --config p/typescript \
        --config p/react --config p/secrets --error .
trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL .
gitleaks detect --source . --redact

# Image
docker build -t fileflow:audit . && trivy image --severity HIGH,CRITICAL fileflow:audit

# Fuzzing long (campagne C3)
cd backend/fuzz && cargo +nightly fuzz run resolve -- -max_total_time=3600

# DAST (voir doc. 13, annexe A)
docker run --rm -v "$PWD:/zap/wrk" ghcr.io/zaproxy/zaproxy:stable \
  zap.sh -cmd -autorun /zap/wrk/zap-plan.yaml

# Chaîne d'approvisionnement
zizmor .github/workflows/
syft fileflow:audit -o cyclonedx-json > sbom.cdx.json
```